

Pay Attention! Human-Centric Improvements of LLM-based Interfaces for Assisting Software Test Case Development

Billy Shi
zs371@cam.ac.uk
University of Cambridge
Cambridge, UK

Per Ola Kristensson
pok21@cam.ac.uk
University of Cambridge
Cambridge, UK

Abstract

Implementing automation testing is difficult and as a consequence there is a growing desire for semi-automated software testing systems with humans in the loop. Leveraging the growth of LLMs, recent research has demonstrated LLMs’ potential to improve performance on test generation, reporting, and bug triaging. However, relatively little work has explored the interactivity issues that emerge in semi-automated LLM-assisted software test case development. To fill this gap, we present two user studies ($N_1 = 16$, $N_2 = 24$) that investigate productivity, creativity, and user attention in three semi-automated LLM-assisted interaction strategies: (1) pre-emptive prompting; (2) buffered response; and (3) guided input. We find that pre-emptively prompting the user significantly enhances branch coverage and task creativity by more than 30% while reducing user’s off-task idle time by up to 48.7%. We conclude by suggesting concrete research directions applying mixed-initiative principles for LLM-based interactive systems for semi-automated software testing.

ACM Reference Format:

Billy Shi and Per Ola Kristensson. 2024. Pay Attention! Human-Centric Improvements of LLM-based Interfaces for Assisting Software Test Case Development. In *The 37th Annual ACM Symposium on User Interface Software and Technology (UIST Adjunct ’24)*, October 13–16, 2024, Pittsburgh, PA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3672539.3686341>

1 Introduction

Practical software testing automation struggles between “the devil and the deep blue sea”—manual testing is inefficient and unreliable, but automation testing is costly and complex. The time to test software accumulates to 50% of the entire software development cycle [6], and the cost to repair defects escalates exponentially as changes are made later [4]. The rising popularity of large language models (LLMs) provides a timely opportunity for LLM-based testing automation. LLMs may assist software testers in developing test cases from specifications to code. Recent research have focused on enhancing LLMs’ model output in test generation [1, 2, 7], reporting [3], and bug triaging [5], and demonstrated that responsive, well-trained models can generate high-quality ideas quickly and accessibly. However, to what degree can LLMs—having been trained on vast but finite datasets—help improve users’ creativity

in software testing? Will the model output solely determine the value proposition for using LLMs, or will factors such as the *LLMs interface designs* also play a vital role?

We take a novel approach and empirically investigate human-centered semi-automated LLM-assisted software testing, which currently lacks a systems design perspective and empirical evidence. In two user studies, we expose participants to LLM assistance while they brainstorm functional test cases for given Python software, and we empirically report their performance in terms of productivity, creativity and attention span. In Study 1 ($N = 16$) we find that the de-facto conversational interaction strategy with an LLM demands excessive attention and inefficient information transfer. This leads us to introduce three new interface modifications: (1) pre-emptive prompting; (2) buffered response; and (3) guided input. We study the effects of these interaction strategies in Study 2 ($N = 24$), which reveals the pre-emptive prompting interaction strategy yields the best performance.

Overall, we make the following contributions:

- (1) We propose nine metrics for studying LLM-assisted interactive software testing tasks, covering three dimensions: (1) productivity; (2) creativity; and (3) attention span.
- (2) We empirically demonstrate that an LLM interface with pre-emptive prompting improves test quality by 33% and creativity performance by 35% on average.
- (3) We distill design implications for LLM-assisted software test case development: (1) recommending the use mixed-initiative principles; (2) discovering effective and efficient motor-control mechanisms for the buffered response strategy; and (3) conjecturing early warnings of user complacency.

2 Overall Approach

We empirically study the effects of LLMs use and their interface modifications via user studies. Participants recruited are university students with engineering or computer science backgrounds, and they all self-report similar experiences with Python and regular (1-3 times a week) use of LLM-based tools (e.g. ChatGPT)¹. To evaluate participants’ performance, we define nine custom metrics in three dimensions—test quality, creativity and attention span—to quantitatively evaluate a user’s performance in a software test case development (STCD) task. A common metric for test quality is function branch coverage (FBC) which rewards test cases test previously untested part of the source code. The number of unique test cases (NUT) is a measure for creativity, and for the attention span, interaction data such as keystrokes and eye tracking are

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST Adjunct ’24, October 13–16, 2024, Pittsburgh, PA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0718-6/24/10

<https://doi.org/10.1145/3672539.3686341>

¹Two participants without any knowledge of ChatGPT had difficulties completing tasks, whose results were excluded.

analyzed. While we do not attempt to generalize these metrics for other LLM interaction tasks, they offer a different perspective for further human-LLM interaction research.

3 Study 1: Interactions with LLMs

Study 1 was a between-subjects design with two conditions. We recruited 16 participants with eight encouraged to use ChatGPT and eight restricted to web search without any generative content. All participants completed two tasks: one 5-minute task to generate test cases for a single-module triangle classifier without access to the software's source code, and another 30-minute task for a multi-module multiple-choice reporting software with access to source code. To increase internal validity, we adopt a simplified definition of a *software test case* to mean a simple scenario statement or assertion statement. For example, "Opening a file which is already opened should fail". The setup consisted of dual monitors with a standard keyboard and mouse, mimicking a software engineer's work desk, and the computers logged all interactions.

Study 1 found that participants using LLMs exhibited a strong 'satisficing' heuristic during information foraging, and their test quality was significantly worsened. With the 'satisficing' heuristic, most participants (7 out of 8) relied on only one iteration of prompting to select answers for the task, neglecting the potential gain of follow-up prompts. The performance loss was despite, on average, spending 48.7% time interacting with LLMs—126% more time than searching on Google, which signaled a poor multi-tasking strategy. For the 30-minute task, participants in the LLM-assisted condition adjusted their multi-tasking strategies to avoid wasting time on the LLM interface and significantly improved their mental workload and their off-task idle time.

4 Study 2: LLM-based Interfaces

Results from Study 1 directly led to three LLM interface design modifications in Study 2: (1) buffered response of the LLM's output, where the user does not receive any LLM output until it is complete; (2) pre-emptive prompting triggered by user interaction, where the LLM will engage without explicit user prompts; and (3) guided prompts that avoid repetitive user input. We studied these three conditions together with a control condition using the familiar ChatGPT-like interface in a within-subjects setup with 24 participants. The buffered response condition turned off the LLM's output streaming, redirecting the participant's attention back to the software IDE. The guided input condition displayed common input phrases, such as "suggest 3 test cases" or "explain this code snippet". The pre-emptive prompting condition connected the software IDE with the LLM model programmatically, such that user interaction behavior—the mouse cursor hovering over a code snippet for more than five seconds, the cursor's insertion point remains fixed for more than five seconds, the mouse cursor selecting a code snippet, and a new test case being manually inserted—became cues for the LLM to suggest previously unsuggested test cases.

Each participant completed four tasks in a designated order balanced by a Latin square. In each task, participants had 7 minutes to develop test cases with one of the aforementioned LLM interfaces. We found that pre-emptive prompting yielded a promising performance improvement, partially due to the trade-off between

keystrokes and mouse clicks, with the latter demanding less user effort. All metrics in test quality and creativity demonstrated significantly higher task performance compared to the control condition ($p < 0.05$ and $r > 0.5$ for all). A linear regression analysis showed that about 20% of the test quality improvement could be explained by this human-centric design modification, and participants also confirmed this perception in a questionnaire and NASA-TLX survey: 50% of participants preferred pre-emptive prompting over other conditions and on average participants rated their perceived performance significantly higher and their mental workload significantly lower for pre-emptive prompting.

The buffered response condition was a double-edged sword as some participants benefited from the buffer to achieve better multitasking while others struggled with the frustration of an extended wait for the system response. Eye gaze data showed a clear difference in attention distribution between the two groups ($p = 0.0005$, $D = 0.136$). Further analysis also found that participants' task performance, which deteriorated due to the prolonged wait for a system response, was partly attributable to their over-reliance on the LLM's output—an early sign of user complacency. The empirical evidence show how design features can have emergent consequences on the user experience beyond the intent of the designer. In this case, an empty LLM interface with a loading bar was not well perceived by participants, nor were they encouraged by the system to focus on the IDE interface.

5 Design Implications and Conclusion

The findings highlight key design guidelines: integrating mixed-initiative principles, designing for user appropriation, and avoiding complacency. Pre-emptive prompting shows promise, providing a foundation for future research on mixed-initiative LLM-based systems that optimize automation level and machine sensitivity. Brief follow-up interviews with two participants revealed a strong preference for opportune interruption and user agency in mixed-initiative LLM systems. Participants overwhelmingly disapproved of autonomous automation, preferring user control due to concerns about loss of agency and perceived complacency.

For the guided input and buffered response conditions, results demonstrate that user's acceptability of these interaction mechanisms was challenged due to a lack of visibility of the system's status and a lack of possibility for early termination. Guided input prompts also lacked the personalization to fully unleash the intended benefits. Future design modifications could consider redirecting the user's attention more proactively—for instance, via an ephemeral interface hiding the LLM interface, and by introducing cautionary reminders about over-trusting the LLM's output. Though one may argue that this problem becomes irrelevant as LLM models respond faster, we believe ephemeral LLM interfaces may also foster better user habits in multi-tasking.

We see two strands of future work: (1) 'how can pre-emptive prompting learn from user inputs to achieve reinforced, opportune interruptions; and (2) what is the role of additional cognitive factors, such as learning and engagement, in the software testing process, and how can they potentially influence LLM-based system design.

References

- [1] Robert Feldt, Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Towards Autonomous Testing Agents via Conversational Large Language Models. arXiv:2306.05152 [cs.SE]
- [2] Soneya Binta Hossain and Matthew Dwyer. 2024. TOGLL: Correct and Strong Test Oracle Generation with LLMs. arXiv:2405.03786 [cs.SE]
- [3] Yuchao Huang, Junjie Wang, Zhe Liu, Yawen Wang, Song Wang, Chunyang Chen, Yuanzhe Hu, and Qing Wang. 2023. CrashTranslator: Automatically Reproducing Mobile Application Crashes Directly from Stack Trace. arXiv:2310.07128 [cs.SE]
- [4] Capers Jones. 1997. *Applied Software Measurement*. McGraw-Hill Companies.
- [5] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2023. Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions. arXiv:2310.15780 [cs.SE]
- [6] Glenford J Myers, Corey Sandler, and Tom Badgett. 2012. *The art of software testing*. John Wiley & Sons.
- [7] Juan Altmayer Pizzorno and Emery D. Berger. 2024. CoverUp: Coverage-Guided LLM-Based Test Generation. arXiv:2403.16218 [cs.SE]